

LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. **AI APPLICATIONS
IN PRACTICE**

6a - UV Package Manager

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

UV

What is UV?

UV is a modern Python packaging toolchain built in Rust. It serves as both a package installer and project management tool designed to be much faster than traditional Python package managers.

Key characteristics:

- Extremely fast package installation
- Compatible with pip's interface
- Supports virtual environments
- Can replace pip, virtualenv, and parts of other tools
- Created by Astral (formerly Suno) as an open-source project

3

Why UV?

Traditional Python package management can be slow and complex. UV addresses these issues by:

- Providing significantly faster installation speeds (10-100x faster than pip)
- Offering a simpler, more consistent user experience
- Supporting reproducible builds through lockfiles
- Maintaining compatibility with the Python ecosystem
- Resolving dependencies in parallel with optimized algorithms

4

Installation

Installing UV is straightforward:

```
# Install via curl (Unix/macOS)
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Or with pip
pip install uv
```

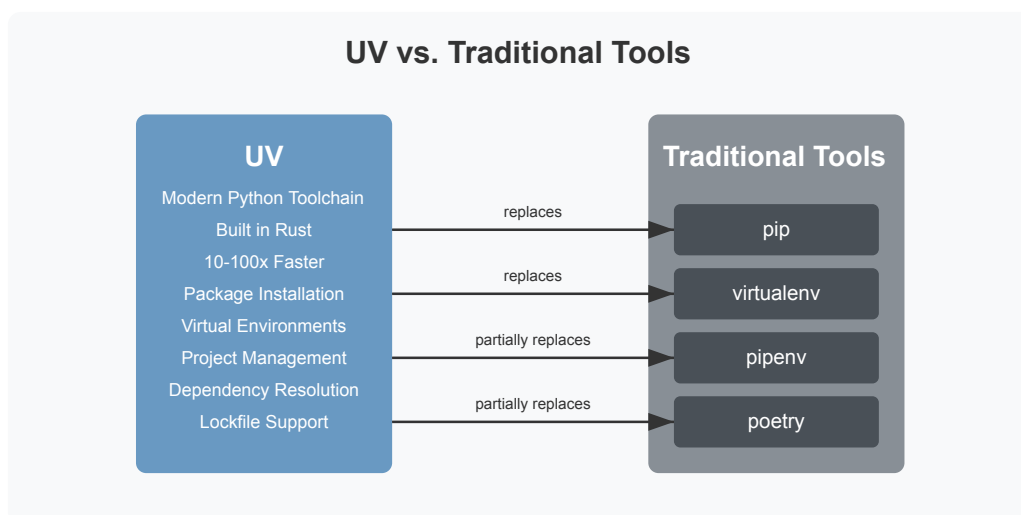
```
# Or on macOS with Homebrew
brew install uv
```

After installation, verify it works:

```
uv --version
```

5

UV vs. Traditional Tools



UV can replace or supplement:

- pip (for package installation)
- virtualenv (for virtual environment management)
- Parts of pipenv and poetry (project management)

6

Basic Commands

UV provides intuitive commands for common tasks:

```
# Install packages
uv pip install requests pandas

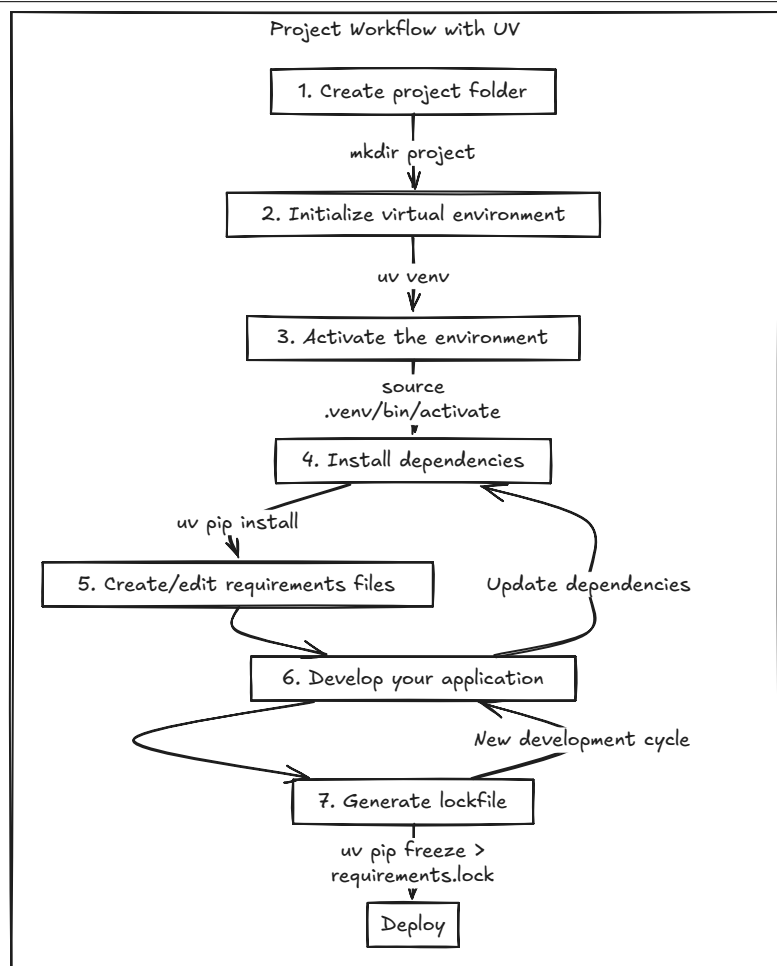
# Create a virtual environment
uv venv

# Activate the environment (same as normal venv)
source .venv/bin/activate # Unix/macOS
.venv\Scripts\activate    # Windows

# Install from requirements file
uv pip install -r requirements.txt
```

7

Project Workflow with UV



Project Workflow with UV

1. Create a new project folder
2. Initialize virtual environment with `uv venv`
3. Activate the environment
4. Install dependencies with `uv pip install`
5. Create/edit requirements files
6. Develop your application
7. Generate lockfile with `uv pip freeze > requirements.lock`

9

Reproducing Environments

UV makes environment reproduction reliable.

This ensures exact package versions across different environments.

Create a lockfile from current environment

```
uv pip freeze > requirements.lock
```

On another machine/environment

```
uv venv
```

```
source .venv/bin/activate
```

```
uv pip install -r requirements.lock
```

10

Practical Example: installing numpy

```
# Create project folder
mkdir project
cd project

# Ensure the version of Python required is installed
# for instance 3.12
uv python install 3.12

# Create and activate virtual environment
uv venv --python 3.12
source .venv/bin/activate

# Install numpy
uv pip install numpy

# Develop the application

# Save dependencies for deploy
uv pip freeze > requirements.txt
```

11

Resources

Official documentation: <https://github.com/astral-sh/uv>

Astral website: <https://astral.sh>

Comparison with other tools: <https://astral.sh/blog/uv>

12